Data Serialization & Schema Standards

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

Our services exchange JSON over HTTP and Pub/Sub, validated with Pydantic models and persisted in Spanner and BigQuery. When a schema changes without regard for older clients, or when types are serialized carelessly, requests that used to work start failing. This guide is the prescriptive standard for evolving schemas compatibly and for serializing the types that most often cause bugs: coerced values, datetimes, and money.

Standards

Backward-compatible schema evolution

Never add a required field to an existing request or response schema — older clients that do not send or expect it will fail validation. Add new fields as *optional with a default*, version the API when a breaking change is unavoidable, and deprecate before removing.

```
class UserV2(BaseModel):
   id: str
   name: str
   # additive, non-breaking: optional with a default
   nickname: str | None = None
   tier: str = "standard"
```

Treat schema changes like API changes: an additive, defaulted field is safe; making a field required, renaming it, or removing it is breaking and needs a new version and a deprecation window.

Validation strictness and coercion

Pydantic v2 is stricter about coercion than v1 (for example it will not silently turn a string into an int). Decide the intended contract explicitly rather than relying on accidental coercion: relax with a validator where the loose behavior is intended, or fix callers to send correct types, and document whichever you choose.

```
class Order(BaseModel):
   # accept "42" or 42 only if that is a deliberate contract
   quantity: int
   @field_validator("quantity", mode="before")
   @classmethod
   def coerce_qty(cls, v):
       return int(v)
```

Datetimes: always timezone-aware UTC

Serialize timezone-aware UTC datetimes as ISO-8601 with an offset. Never serialize a naive datetime and never call `datetime.now()` without a timezone — different consumers will read it as local time and

be off by hours. Convert to local only at display time.

```
from datetime import datetime, timezone
ts = datetime.now(timezone.utc)          # aware, UTC
ts.isoformat()                           # '2026-07-10T12:00:00+00:00'
```

Money: never binary floats

Binary floats cannot represent decimal fractions exactly, so summing them drifts by a cent. Use `decimal.Decimal` (or integer minor units), quantize to the currency precision, and never sum floats for financial values.

```
from decimal import Decimal
total = Decimal("0.10") + Decimal("0.20")   # Decimal('0.30')
```

Common failures and fixes

- Adding a required field breaks older clients → schema drift; add new fields as optional with defaults, version the API, and deprecate before removing (BUG-1062).
- Requests that used to succeed now fail validation after a library upgrade → Pydantic v2 stricter coercion; choose the contract explicitly with validators or fix callers, and document the stricter schema (BUG-1058).
- Timestamps are wrong across regions → naive datetimes serialized without tzinfo; store and serialize tz-aware UTC ISO-8601 and convert to local only at display (BUG-1059).
- Totals off by a cent → money computed with binary floats; use `Decimal` or integer minor units and quantize to currency precision (BUG-1060).

Checklist

- New fields are optional with defaults; no new required field on an existing schema.
- Breaking changes go behind a new API version with a deprecation window.
- Validation strictness is a deliberate, documented contract, not accidental coercion.
- All datetimes are tz-aware UTC, serialized as ISO-8601 with offset.
- Money uses `Decimal` or integer minor units, quantized to currency precision.
- Schema shared between client and server so both stay in sync.

Related incidents

BUG-1062, BUG-1058, BUG-1059, BUG-1060